03. Procesamiento de la Malla 3D (`generar_malla_offline.py`)

Este es el núcleo matemático del proyecto. Explica cómo transformar millones de puntos dispersos de una nube (`.ply`) en una superficie continua que los motores de juegos o renders pueden interpretar como un objeto sólido texturizado.

1. El Problema de las Nubes de Puntos

El escáner produce vértices. Sin embargo, no hay "polígonos" o "caras" (triángulos) que los unan. Los puntos por sí solos no rebotan la luz en Blender de manera nativa ni se pueden importar a motores como Unreal Engine como un "Mesh". Se necesita un proceso de triangulación.

2. Proyección 2.5D (De 3D a Pixeles 2D)

Para construir la superficie, el script lee la matriz intrínseca de la cámara desde el archivo JSON: - f_x, f_y: Longitudes focales. - c_x, c_y: Centro óptico.

Aplica la fórmula matemática de retro-proyección de Pinhole (Agujero de alfiler):

```
u = (X * fx / Z) + cx
v = (Y * fy / Z) + cy
```

Esto "aplasta" todos los puntos 3D contra una foto plana bidimensional. Al estar ahora en 2D, es mucho más sencillo conectar los puntos sin cometer errores que crucen geometrías complejas.

3. Triangulación de Delaunay

Utilizando `scipy.spatial.Delaunay` , el script conecta los puntos 2D (U, V) creando una red de triángulos contigua. La propiedad fundamental de Delaunay es que maximiza el ángulo mínimo de los triángulos (evitando polígonos extremadamente largos y delgados que causan artefactos de renderizado).

4. Limpieza de Artefactos de Profundidad (Filtrado de Aristas)

El problema de la proyección 2D es que puede conectar el hombro de una persona con la pared que está a 3 metros atrás de él, porque visualmente desde la cámara están contiguos. Para evitar estas "telas de araña" 3D:

```
# Se obtienen las coordenadas Z y longitudes de las 3 aristas
l0 = np.linalg.norm(v0 - v1, axis=1)
# ... se eliminan los triángulos cuya arista sea > 5cm (0.05 m)
valid_mask = (l0 < 0.05) & (l1 < 0.05) & (l2 < 0.05)
```

5. Mapeo UV y Texturizado

Los triángulos supervivientes se ensamblan en un `o3d.geometry.TriangleMesh`. Para darles la textura de la foto original (`captura_color.jpg`), las coordenadas (U, V) obtenidas en el Paso 2 se normalizan entre `0.0` y `1.0`. Debido a estándares gráficos, el eje V (vertical) debe ser invertido (`1.0 - (v / h)`) para que coincida con el estándar `.obj`.

Finalmente, se calculan las normales de los vértices (para iluminación correcta) y se exporta como `malla_texturizada.obj`.